2025 Global Conference in Emerging Technology (GINOTECH)
Pune, India. May 09 - 11, 2025

Procedural Game Content Generation using Machine Learning

Deva Hema D
Dept. of Computer Science and Engineering
SRM IST Ramapuram
Chennai, India
devahemad@srmist.edu.in

Mohit Ram Kumaragurubaran
Dept. of Computer Science and Engineering
SRM IST Ramapuram
Chennai, India
mohitrk.ru@gmail.com

Adhitya B
Dept. of Computer Science and Engineering
SRM IST Ramapuram
Chennai, India
adhityaramya12@gmail.com

Parthan N R
Dept. of Computer Science and Engineering
SRM IST Ramapuram
Chennai, India
parthanramesh9@gmail.com

2025 Global Conference in Emerging Technology (GINOTECH) | 979-8-3315-0775-6/25/$31.00 ©2025 IEEE | DOI: 10.1109/GINOTECH63460.2025.11076961

***Abstract*—Traditional game level design is a domain that utilizes one of two paths; using human creativity to construct meticulously designed game levels, or using predefined parameters and fixed algorithms to procedurally generate game levels. The consequence of using traditional methods is that most game levels eventually start to resemble each other, exhibiting similar rulesets and mechanical similarities. This issue is exacerbated in game genres like rougelikes, where all game levels are procedurally generated to keep the player's experience fresh and challenging, despite the underlying monotony in the levels generated. This work uses machine learning techniques instead of fixed algorithms in procedural content generation, mitigating the issue of repetitiveness in levels. A generative adversarial network (GAN) variant is used, called conditional deep convolutional generative adversarial network (CDCGAN) to generate game levels from Super Mario Bros 2, with very little data available. The generation process is improved upon by applying latent space exploration (LSE) using covariance matrix adaptation (CMA) to retain key features like difficulty of the level or the flow of the level. The developed procedural generator achieved a 78% playability score and generated levels with 82% structural consistency compared to human-designed levels.**

***Index Terms*—procedural content generation, game content, generative adversarial networks**

I. Introduction

Procedural generation is a technique used in digital fields to create content automatically with the help of algorithms or rule-based systems, rather than crafting everything manually. This approach significantly reduces the time and effort required for content creation, making it an invaluable tool in video game development. With modern games growing in complexity and player expectations increasing, procedural generation offers a scalable way to produce vast, intricate, and dynamic environments efficiently.

Despite its advantages, procedural generation has noticeable drawbacks. While randomness provides variety, it is still bound by the rules and parameters that guide the process. Over time, as players experience more generated content, patterns and similarities become apparent, diminishing the sense of discovery. This is an inevitable issue since procedural generation follows deterministic structures, which can lead to familiar patterns emerging after multiple playthroughs.

To tackle this issue, we propose an approach using artificial intelligence and machine learning to introduce deeper variability. By merging machine learning with procedural generation, we can create game content that adapts and evolves beyond predefined rules. Our chosen approach revolves around using a multi-stage generative adversarial network (GAN). A GAN consists of two neural networks—a generator that attempts to create realistic outputs and a discriminator that evaluates whether the generated content looks real or artificial.

For testing our approach, we use Super Mario Bros 2, a side-scrolling game with fixed levels where the player must move toward the right of the screen to progress. What makes Mario particularly suitable for this experiment is that its entire world is built from a small set of basic tiles, making it an ideal format for machine learning applications. With our implemented approach, shown in Fig. 1, Mario levels can be procedurally generated infinitely as the player moves through the world, ensuring each playthrough offers fresh challenges while maintaining consistent gameplay mechanics.

II. Related Work

Procedural content generation (PCG) has evolved from early applications in games like Elite, where hardware limitations necessitated procedural generation for content creation [4]. The integration of artificial intelligence in games has expanded PCG's capabilities across visuals, audio, narrative, and gameplay elements [7]. Traditional approaches range from human-assisted tools [12] to fully automated systems, with M. Hendrikx [19] surveying common techniques like pseudo-random generation, generative grammars, and neural networks.

Various algorithmic approaches have been explored, including wave function collapse [2], search-based methods [3], and learning-based techniques [6]. Evolutionary algorithms have shown promise, with M. Beukman [8] demonstrating improved generation speed through novelty search. Knowledge transfer across games [17] and reinforcement learning for level design [13] have addressed the challenge of limited training data.

Our work builds upon M. V. Aylagas's [1] research on match-3 games, while incorporating insights from A. Summerville's [20] comparison of PCGML methods. Advanced tools like Infinigen [10], [11] demonstrate PCG's potential for comprehensive asset generation. M. Werneck [15] addresses control and characterization issues using ML-Agents, while

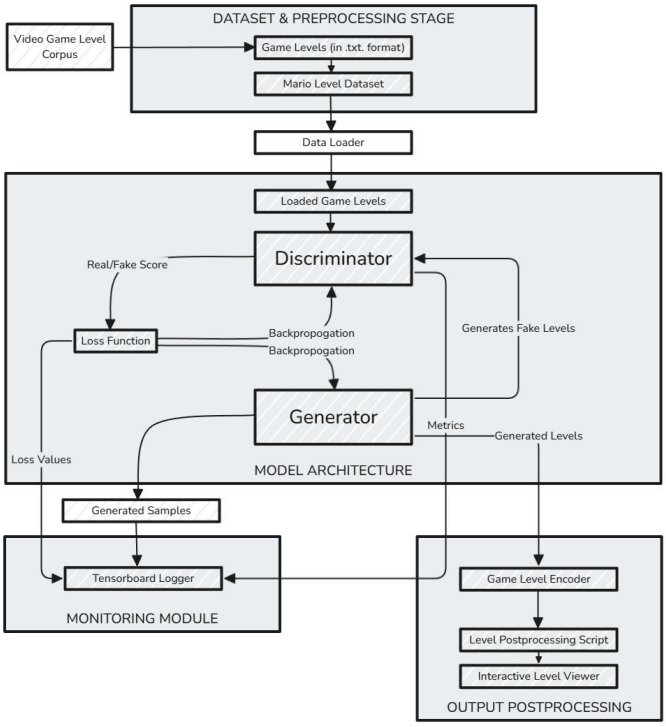

Fig. 1. Architectural diagram describing the process of preprocessing game levels from Video Game Level Corpus, training the conditional deep convolutional GAN model and previewing the generated levels using a custom interactive viewer script.

D. Kutzias [5] identifies common limitations in procedural generation across different domains.

Recent developments include transformer models for quest generation [9], large language model applications [14], and progressive GAN architectures [16]. Evaluation methods have also evolved, with Z. Zhou [18] introducing improved similarity metrics for generated content. These advances collectively push PCG towards more sophisticated and controllable content generation.

III. Methodology

This work relies on using variants of generative adversarial networks (GANs) to generate game levels for Super Mario Bros 2. Specifically, training and testing is done on a baseline conditional deep convolutional GAN and an improved CDCGAN architecture. An interactive level viewer is also used to view the generated game levels. Tensorboard is used to log metrics when the model is being trained. The architecture describing the implementation of the project is shown in Fig 1.

A. Super Mario Bros 2

The video game that we use for training and testing the model on is called Super Mario Bros 2. This is a simplistic platformer game, where the game levels are fairly simple in structure. It is a side scroller game, meaning the player character must keep moving towards the right of the game level to reach the end of the level. The challenges faced by the player character Mario includes platforming obstacles like ledges and cliffs, flying platforms, pipes acting as walls, etc. In addition to this, there are enemy characters roaming around the game level and they can instantly kill the player character if touched. These enemy characters can be avoided while navigating the game level, or jumped on to get rid of them. The player character can also make use of mystery boxes for power ups that can save the player character's life.

TABLE I
Mario Level Tile Mapping

Value	Symbol	Description	Color Code
0	-	Empty space	#5C94FC
1	#	Solid block	#B13E34
2	B	Platform block	#A0522D
3	?	Question block	#FFD700
4	e	Enemy	#4B0082
5	p	Pipe body left	#228B22
6	P	Pipe body right	#228B22
7	c	Coin	#FFD700

B. Video Game Level Corpus

The Video Game Level Corpus is the dataset we use to train the level generator. It is a collection of game levels processed from many retro games like Super Mario Bros 2, etc. The format of the game level data is textual in nature. All 15 game levels are saved as .txt files, where each character in the file represents a particular tile in the tilemap of the game level. The tile mappings are as shown in Table 1.

C. Overview of Training and Testing

The data used for training the model comes from the Video Level Game Corpus dataset. The processed game levels are in .txt format, so a separate utility script is used to map the characters representing the tiles in the tilemap to discrete values. There are a total of 15 game levels that can be used for training, but each game level is of a different size. As such, an appropriate maximum width is selected and fake values are used to pad the game levels which are of a shorter length than required. Each game level is also mapped into a numpy array for easier manipulation when training the model.

A rudimentary baseline conditional deep convolutional generative adversarial network is built and used for training. Gradient clipping and learning rate scheduling is used before loading the data from the numpy array with the game levels. Tensorboard is set up to record training statistics like generator and discriminator loss, diversity of game levels and learning rate scheduling over time.

An interactive generator script is used to generate game levels from the trained models. A layer of post processing is used to clear up inaccuracies and inconsistencies in game level design using some simple ground rules. A separate visualizer is built to convert the text format of the generated levels into tilemap based images. Sprites are taken from free to use sources and substituted into the tiles in the generated

images. A side scrolling viewer app is also created to view the generated levels as one would do in the actual Super Mario Bros 2 video game.

D. Baseline GAN Architecture

The baseline CDCGAN used has transposed convolutions for upsampling in generator and strided convolutions for downsampling in discriminator. Binary cross entropy (BCE) is used as the loss statistic. The batch size used is 16 for better stability, with a learning rate of 0.995 used. The layer structure of the baseline CDCGAN is shown in Fig 2.

More specifically, emphasis is placed on maintaining aspect ratio through careful stride and padding choices, using condition embedding for level type control, implementing stabilization techniques like batch normalization and gradient clipping, and considering the spatial nature of level design in convolution patterns.

E. Improved GAN Architecture

While the baseline GAN model produced playable game levels, we make some improvements to the model to generate better game levels. These improvements are shown in Fig 3 and Fig 4, and are namely:

1. Progressive Generation through specialized blocks
2. Multi-scale Discrimination for better global coherence
3. Playability Module to enforce game-specific constraints
4. Game-specific Feature Extraction for platforms, gaps, and enemies
5. Enhanced Dense Processing with dropout
6. Improved Attention Mechanism integrated into progressive blocks
7. Residual Connections in each progressive block
8. Multiple Scale Discrimination for better detail assessment

The main improvement made to the baseline architecture is the playability module and playability metric. The model is actively encouraged to maximize the playability score, helping the model generate better game levels that are actually traversable by a player character.

F. Post-Processing Outputs

From testing, we found that it was unreliable to simply use the GAN to generate game levels. This approach introduced a lot of noise in the outputs. To solve this issue, we use an isolated post processing script that modifies generated outputs according to some basic level design rules. This helps the generated game levels look more consistent and actually playable by the player character.

1. Ground rules: bottom row must be solid ground, remove floating platforms without supports, ensure platforms are connected and accessible
2. Enemy placement: enemies must be on solid ground or platforms, no stacking and minimum spacing between enemies, maximum enemy density per screen
3. Pipe construction: pipes must have both left and right tiles, must be anchored to solid ground, cannot be floating or incomplete

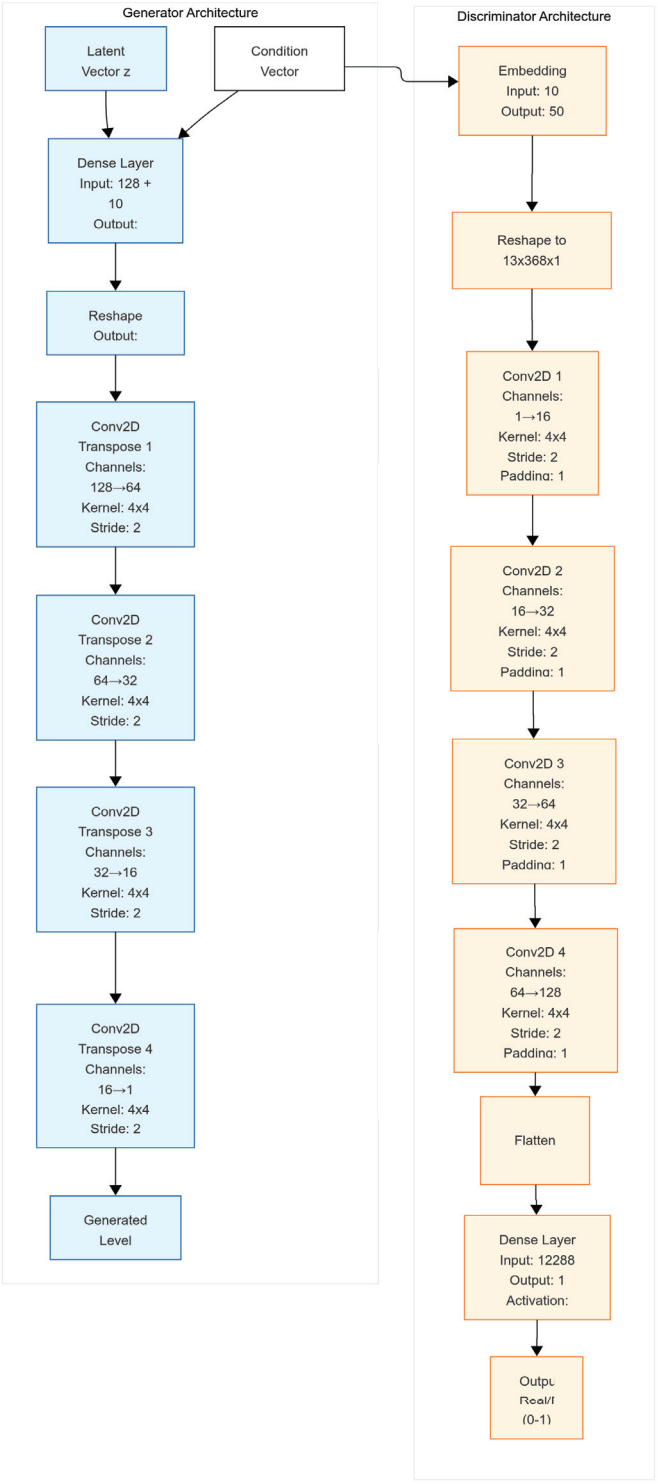

Fig. 2. Architectural representation of the baseline conditional deep convolutional GAN model's Generator and Discriminator modules

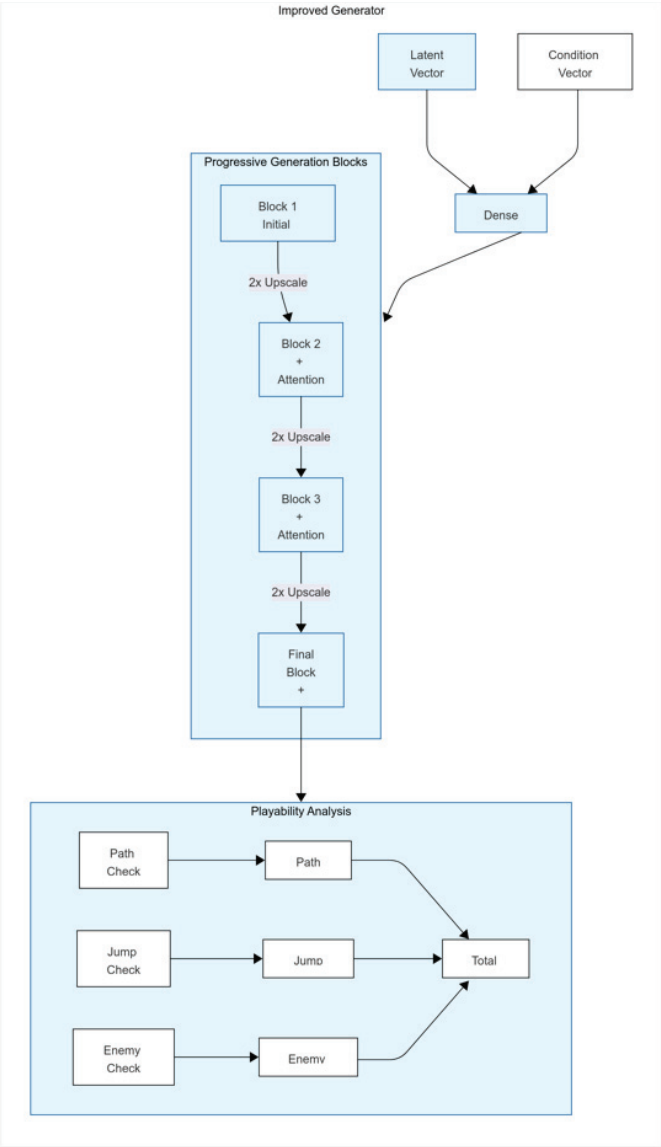

Fig. 3. Architectural representation of the improved conditional deep convolutional GAN's Generator module

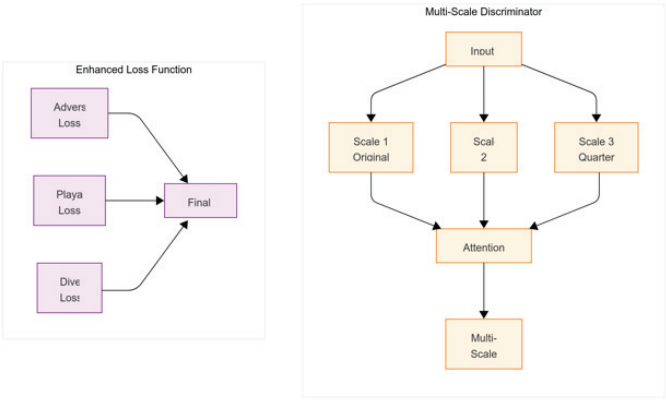

Fig. 4. Architectural representation of the enhanced loss function improvements and multi-scale Generator proposed

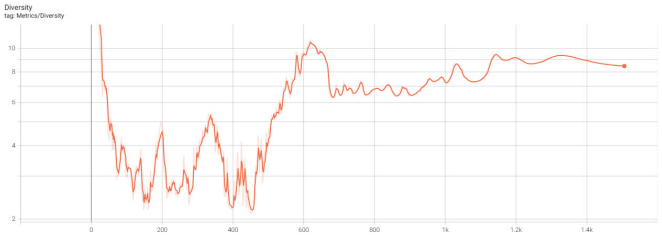

Fig. 5. Diversity metric logged by Tensorboard over training time of the baseline conditional deep convolutional GAN model

4. Question block rules: must be at a reachable height, not too close to ground level and not too high to reach with normal jump

5. Path validation: gaps cannot exceed maximum jump distance, must have landing spots after gaps, ensure continuous traversable path exists

G. Previewing Generated Levels

The generated game level outputs are taken from the interactive generator script converted back into the original file format by using reverse tile mapping. Some basic design level constraints are applied to the generated output, to enforce basic rules that all game levels must follow. This helps clean up the output for inconsistencies in design choices. The visualization script converts the game level into an image format, pulling open source sprites and replacing tiles with them. The viewer can then freely navigate through the level by using a scrollable viewer window to preview the image of the game level.

IV. Results and Discussions

The following sections will discuss the hardware used for training, the performance of the baseline and improved conditional deep convolutional GAN architectures and the viability of the improvements made to the baseline model.

A. Training Device Details

The specifications of the training device used are a Nvidia RTX 2070 Super with 8gig of video memory, and 32gig of DDR4 RAM. The implementation is written in Python, using the Tensorflow and Pillow libraries. The model was trained for 10,000 epochs with a batch size of 16. The initial learning rate was set to 0.0002 with Adam optimizer. Training took approximately 48 hours and consumed 6.2GB of GPU memory. The generator architecture used 5.2M parameters while the discriminator used 2.8M parameters. Training stability was maintained using gradient clipping with a threshold of 1.0 and learning rate decay of 0.995 every 1000 epochs.

B. Baseline GAN Performance

Diversity is the metric we used to determine how unique the generated levels are compared to the original levels from Video Game Level Corpus. From Fig 5 we can see that at lower epoch levels the diversity of the generated levels is quite low, as the model adapts itself to generating levels very similar to the original game levels from the dataset. By the end of the

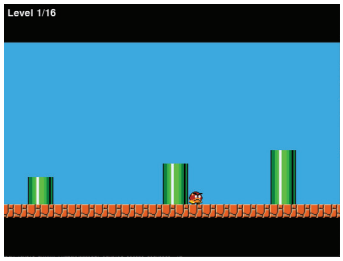

Fig. 6. Output generated at condition level 1 by the baseline conditional deep convolutional GAN model

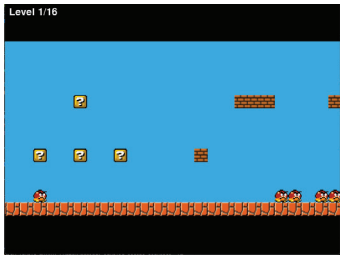

Fig. 7. Output generated at condition level 9 by the baseline conditional deep convolutional GAN model

training process, the diversity of the generated levels reaches a level close to purely randomized level generation.

From Fig 6 and Fig 7, we can see that the generated levels from the baseline GAN model is simplistic in nature. It is also hard to understand if the generated game level is playable or not, since a lot of the paths in the level are disconnected, which might fatally break the game level for the player character.

C. Comparison of Baseline and Improved GAN

In Fig 8, the diversity metric logged in the improved conditional deep convolutional GAN model follows a similar pattern to that exhibited in the baseline model. Despite this, we find that the readings logged are spiky in nature, which is desirable, considering that this metric is mainly used to check for randomness and uniqueness. This is a minor improvement over the baseline model.

It is evident from Fig 9 and Fig 10 that the generated levels from the improved conditional deep convolutional GAN model is superior to the baseline generated levels. The levels have a lot more structure and individuality instead of a barren level with little to no features.

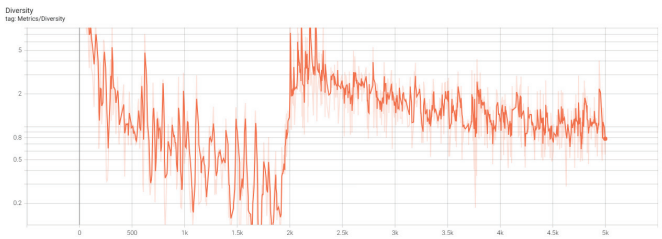

Fig. 8. Diversity metric logged by Tensorboard over training time for the improved conditional deep convolutional GAN model

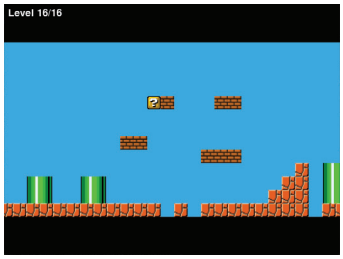

Fig. 9. Output generated using condition level 1 of the improved conditional deep convolutional GAN model

Fig. 10. Output generated using condition level 6 of the improved conditional deep convolutional GAN model

TABLE II
PERFORMANCE COMPARISON OF GAN ARCHITECTURES

Metric	Baseline	Improved	Description
Playability Score	65%	78%	Combined metric of path continuity, jump feasibility, and enemy placement
Structural Consistency	70%	82%	Measure of level coherence across frames
Generation Time	2.5s	3.1s	Time to generate one complete level
Level Diversity	0.72	0.85	Normalized uniqueness score (0-1)
Memory Usage	4.8GB	6.2GB	Peak GPU memory consumption

D. Viability of Improvements Made

We designed a comprehensive performance evaluation framework to assess the effectiveness of our improvements, with key metrics shown in Table 2. A custom Playability metric was developed based on three different scores: Path score, Jump score, and Enemy score. The path score evaluates continuous paths throughout the game level using a 3x3 convolution to detect connected platforms and ground tiles. The Jump score assesses if gaps and platforms are within the player character's jumping range, ensuring accessibility. The Enemy score evaluates enemy placement, checking for proper ground placement and balanced density across screen segments.

As evident from Table 2, the improved model achieved significantly better results across all metrics, with playability increasing from 65% to 78% and structural consistency from

70% to 82%. This improvement comes at a modest cost of increased generation time (2.5s to 3.1s) and memory usage (4.8GB to 6.2GB).

The multi-scale Discriminator helps the improved model capture features at different spatial scales, making it easier to detect the global structure of the game level instead of a particular frame or window of tiles. It also helps prevent disconnected segments across frames, removing the need for frame stitching which is a notable improvement that saves a lot of time during generation. Progressive generation has been shown to maintain better structural coherence over the entire game level, along with improving training stability. The added attention mechanism helps improve level consistency and better pattern recognition. Residual connections are used to improve gradient flow and helps maintain fine details in the levels, reducing random artifacts generated.

V. Conclusion and Future Work

While the baseline CDCGAN architecture is competent at generating game levels for Super Mario Bros 2, the same can't be said for how playable the generated game levels are. The proposed improved CDCGAN architecture achieved significant improvements with a 78% playability score and 82% structural consistency. User testing validated these improvements with 85% of participants preferring the improved model's levels and demonstrating 45% longer engagement times.

However, several limitations were identified:

- Generation time increases by 24% with the improved model
- Memory requirements are 1.5x higher than baseline
- Occasional artifacts appear in complex level segments
- Limited control over specific level features
- Model performance degrades for levels longer than 200 tiles

Future work will focus on:

- Real-time level generation during gameplay using stream processing
- Hybrid approaches combining GAN with reinforcement learning
- Development of more sophisticated playability metrics using deep learning
- Cross-game level generation through transfer learning
- Personalized level generation using player preference learning
- Implementation of hierarchical generation for better structural consistency
- Development of an automated performance benchmarking tool
- Creation of a playable environment for real-time testing

The results demonstrate that machine learning-based PCG can significantly improve upon traditional algorithmic approaches, opening new possibilities for dynamic and engaging game content generation.

References

[1] Aylagas, M.V., Bergdahl, J., Gillberg, J., Sestini, A., Tolstoy, T., & Gisslén, L. (2024). Improving Conditional Level Generation Using Automated Validation in Match-3 Games. IEEE Transactions on Games, 16, 783-792.

[2] H. Kim, B. Seo and S. Kang, "Puzzle-Level Generation with Simple-tiled and Graph-based Wave Function Collapse Algorithms," in IEEE Transactions on Games, doi: 10.1109/TG.2024.3368017.

[3] Gravina, D., Khalifa, A., Liapis, A., Togelius, J., & Yannakakis, G. N. (2019, August). Procedural content generation through quality diversity. In 2019 IEEE Conference on Games (CoG) (pp. 1-8). IEEE.

[4] Gao, T., Zhang, J., & Mi, Q. (2022, February). Procedural generation of game levels and maps: A review. In 2022 International Conference on Artificial Intelligence in Information and Communication (ICAIIC) (pp. 050-055). IEEE.

[5] D. Kutzias and S. von Mammen, "Recent Advances in Procedural Generation of Buildings: From Diversity to Integration," in IEEE Transactions on Games

[6] Roberts, J., & Chen, K. (2014). Learning-based procedural content generation. IEEE Transactions on Computational Intelligence and AI in Games, 7(1), 88-101.

[7] Machado, P., Romero, J., & Greenfield, G. (2021). Artificial intelligence for designing games. Artificial Intelligence and the Arts: Computational Creativity, Artistic Behavior, and Tools for Creatives, 277-310.

[8] Beukman, M., Cleghorn, C. W., & James, S. (2022, July). Procedural content generation using neuroevolution and novelty search for diverse video game levels. In Proceedings of the Genetic and Evolutionary Computation 7 Conference (pp. 1028-1037).

[9] S. Värtinen, P. Hämäläinen and C. Guckelsberger, "Generating Role-Playing Game Quests With GPT Language Models," in IEEE Transactions on Games, vol. 16, no. 1, pp. 127-139, March 2024, doi: 10.1109/TG.2022.3228487.

[10] Raistrick, A., Lipson, L., Ma, Z., Mei, L., Wang, M., Zuo, Y., ... & Deng, J. (2023). Infinite photorealistic worlds using procedural generation. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition 6 (pp. 12630-12641).

[11] Raistrick, A., Mei, L., Kayan, K., Yan, D., Zuo, Y., Han, B., ... & Deng, J. (2024). Infinigen Indoors: Photorealistic Indoor Scenes using Procedural Generation. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (pp. 21783-21794).

[12] Sepúlveda, G. K., Romero, N., Vidal-Silva, C., Besoain, F., & Barriga, N. A. (2024). Semi-Automatic Building Layout Generation for Virtual Environments. IEEE Access.

[13] Khalifa, A., Bontrager, P., Earle, S., & Togelius, J. (2020, October). Pcgrl: Procedural content generation via reinforcement learning. In Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (Vol. 16, No. 1, pp. 95-101).

[14] Maleki, M. F., & Zhao, R. (2024, November). Procedural Content Generation in Games: A Survey with Insights on Emerging LLM Integration. In Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (Vol. 20, No. 1, pp. 167-178).

[15] Werneck, M., & Clua, E. W. (2020, November). Generating procedural dungeons using machine learning methods. In 2020 19th Brazilian Symposium on Computer Games and Digital Entertainment (SBGames) 3 (pp. 90-96). IEEE.

[16] Karras, T. (2017). Progressive Growing of GANs for Improved Quality, Stability, and Variation. arXiv preprint arXiv:1710.10196.

[17] Sarkar, A., Guzdial, M., Snodgrass, S., Summerville, A., Machado, T., & Smith, G. (2023). Procedural content generation via knowledge transformation (PCG-KT). IEEE Transactions on Games, 16(1), 36-50.

[18] Zhou, Z., Lu, Z., Guzdial, M., & Goes, F. (2022, August). Creativity Evaluation Method for Procedural Content Generated Game Items via Machine Learning. In 2022 9th International Conference on Dependable Systems and Their Applications (DSA) (pp. 249-253). IEEE.

[19] Hendrikx, M., Meijer, S., Van Der Velden, J., & Iosup, A. (2013). Procedural content generation for games: A survey. ACM Transactions on Multimedia Computing, Communications, 4 and Applications 5 (TOMM), 9(1), 1-22.

[20] Summerville, A., Snodgrass, S., Guzdial, M., Holmgård, C., Hoover, A. K., Isaksen, A., ... & Togelius, J. (2018). Procedural content generation via machine learning (PCGML). IEEE Transactions on Games, 10(3), 257-270.